Smart Campus Resource Booking System

Mid-Semester Evaluation Report
Design, Implementation Status, and Planned Validation

[Dhruv Bihani] – [1024030736]

[Aditya Srivastava] – [1024030745]

Group: [3C53]
BE Third Year, Computer Science and Engineering

Faculty Advisor: [Dr.Paramveer Kaur]

UCS503 – [Software Engineering]
Department of Computer Science and Engineering
Thapar Institute of Engineering and Technology, Patiala

Mid-Semester Evaluation

Contents

1 Introduction **1**

1.1 Background and Context 1

1.2 Problem Statement 1

1.3 Project Objectives 1

1.4 Scope 2

2 Methodology and Design **3**

2.1 Development Methodology 3

2.2 System Architecture 3

2.2.1 High-Level Architecture 4

2.3 Technical Stack 4

2.4 Database Design 4

2.5 Implementation Details 5

2.5.1 Frontend Module 5

2.5.2 Java Backend Module 5

2.5.3 Booking Workflow 5

2.5.4 Authentication and Access Control 6

2.6 Division of Responsibilities 6

3 Results and Evaluation **7**

3.1 Current Implementation Status 7

3.2 Testing Methodology 7

3.3 Evaluation and Current Limitations 8

3.4 Challenges and Mitigation Plan 8

4 Conclusion **10**

4.1 Summary of Work 10

4.2 Future Work 10

4.3 Final Remarks 10

Chapter 1

Introduction

1.1 Background and Context

Campus activities often require students and staff to identify a suitable resource, check its availability, and coordinate its use. A centralized resource-booking application can provide a common interface for browsing resources and requesting time slots while maintaining booking records in a database.

The Smart Campus Resource Booking System is being developed as a web application for managing campus-resource booking workflows.

At the time of this mid-semester report, the Oracle database is connected, while the frontend, Java backend, and authentication workflow are partially implemented. This report distinguishes current progress from planned functionality.

1.2 Problem Statement

The project addresses the need for a structured digital workflow through which a user can view campus resources, identify a suitable slot, submit a booking request, and review its outcome.

The system must coordinate the user interface, server-side validation and processing, and persistent database records. A key challenge is ensuring that frontend and backend data formats agree and that booking operations respect slot availability.

1.3 Project Objectives

The main objectives of the project are:

1. Develop a web interface for browsing campus resources and initiating bookings.

2. Implement Java server-side components to receive requests, validate input, apply booking rules, and communicate with the database.

3. Use Oracle Database to persist relevant application records.

4. Implement and integrate user authentication and access control.

5. Validate the application using documented functional and integration tests before final submission.

These are project objectives, not a claim that every feature is complete.

1.4 Scope

The current scope focuses on the resource and slot booking workflow, Java backend, Oracle persistence, and authentication.

Additional administration, notifications, reporting, concurrency safeguards, and production deployment can be refined according to the final requirements.

Chapter 2

Methodology and Design

2.1 Development Methodology

The application is being developed incrementally. Frontend and backend work proceed in parallel, with integration required to connect user-facing screens to server-side operations.

The database connection provides a persistence foundation. The next priority is to verify complete user journeys across all layers rather than treating an isolated page or class as an end-to-end feature.

2.2 System Architecture

The intended request flow is:

1. A user interacts with the frontend to view resources or submit a booking request.
2. The frontend sends a request to a Java web endpoint.
3. A Servlet receives the request and passes work to the service layer.
4. The service layer validates input and applies business rules.
5. A Data Access Object (DAO) uses JDBC to read or update Oracle records.
6. The backend returns a response for the frontend to display.

This is the target architecture. Since the frontend, backend, and authentication are partially complete, each connection must be verified against the running application before it is described as implemented.

2.2.1 High-Level Architecture

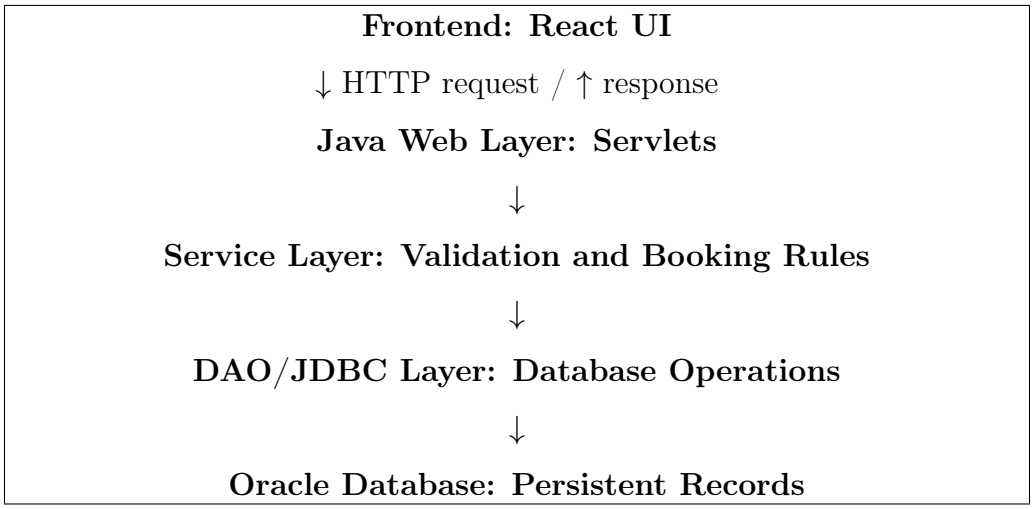


Figure 2.1: Target high-level architecture of the Smart Campus Resource Booking System. Verify each connection against the submitted code.

2.3 Technical Stack

The current development setup uses the following technologies:

- **Frontend:** React and Vite.
- **Backend:** Java, Java Servlets, and JDBC.
- **Database:** Oracle Database.
- **Servlet Container:** Apache Tomcat 9.
- **Build Tool:** Apache Maven.
- **Development Tools:** Visual Studio Code and command-line terminal.

Confirm that each technology is retained in the final submitted implementation.

2.4 Database Design

The database is intended to persist information required for users, campus resources, time slots, and bookings.

The exact table names, columns, constraints, and relationships should be taken from the current Oracle schema.

Entity Area	Purpose
Users	User identity and account-related data.
Resources	Campus facilities or resources that can be booked.
Slots	Time slots associated with resources and their availability.
Bookings	A user's request to reserve a slot and its status.

Table 2.1: Conceptual database entities. Confirm exact schema names and relationships.

2.5 Implementation Details

2.5.1 Frontend Module

The frontend is being developed using React.

Its intended responsibilities include presenting resources, allowing users to navigate to a booking workflow, collecting input, and displaying backend results.

The frontend is partially implemented. Any screen that currently uses sample data or browser storage must be labelled as a prototype rather than proof of database-backed functionality.

2.5.2 Java Backend Module

The backend is being developed using Java web technologies.

Its intended responsibilities include handling HTTP requests, validating request data, applying booking rules, invoking database operations, and returning a response for the frontend.

The backend is partially implemented. The final report should name only the Servlets, service classes, DAO classes, and endpoints verified in the current source tree.

2.5.3 Booking Workflow

The intended workflow is:

1. Receive the selected resource and slot information.
2. Validate required values and identifier formats.
3. Check whether the selected slot is eligible for booking.
4. Create or update the booking through the database-access layer.
5. Return a success or failure result for display in the frontend.

Document status transitions and duplicate-booking protection only after implementation and testing.

A simple availability check before insertion may not prevent simultaneous requests from competing for the same slot. Transaction or database-level safeguards should be considered.

2.5.4 Authentication and Access Control

Authentication is partially implemented.

The intended workflow is to verify credentials, establish an authenticated session or equivalent state, and restrict protected operations where required.

Before final submission, verify successful login, invalid credentials, session handling, logout, and access to protected pages or endpoints.

Passwords must not be stored or logged in plain text.

2.6 Division of Responsibilities

Team Member	**Primary Responsibility**
Backend Developer	Java request handling, service/business logic, JDBC/DAO operations, Oracle integration, authentication integration, backend debugging, and endpoint testing.
Frontend Developer	React components and pages, navigation, forms, input feedback, resource/booking display, and frontend-to-backend API integration.
Both Members	Requirements, architecture, database contract, integration testing, demonstration preparation, documentation, and final verification.

Table 2.2: Division of responsibilities. Adjust if the actual work split differs.

Chapter 3

Results and Evaluation

3.1 Current Implementation Status

Area	Status	Evidence / Next Action
Frontend	Partial	Demonstrate pages that run and identify sample-data or unfinished screens.
Java Backend	Partial	Verify build, deployed context path, and endpoints used in the demonstration.
Oracle Database	Connected	Show a successful read/write operation or other direct evidence.
Authentication	Partial	Demonstrate only verified behavior; test invalid credentials and protected routes.
End-to-End Integration	To be verified	Complete one user journey from frontend request to database and back.

Table 3.1: Mid-semester status based on the team's current assessment.

3.2 Testing Methodology

Tests should record input, expected outcome, actual outcome, and pass/fail result. The following are proposed tests, not claims of tests already passed.

ID	Test Case	Expected Result	Result
T1	Start frontend and open landing page.	Page loads without a blank screen or console error.	Not recorded
T2	Build and deploy Java application.	Maven build succeeds and Tomcat deploys the expected context.	Not recorded
T3	Read a resource or slot through the backend.	Response corresponds to a database record.	Not recorded
T4	Submit a valid booking.	Booking persists and the returned status is displayed.	Not recorded
T5	Submit missing or invalid booking input.	Request is rejected with a controlled error.	Not recorded
T6	Attempt to book an unavailable slot.	Request is rejected and no invalid booking is created.	Not recorded
T7	Try valid and invalid login credentials.	Valid credentials follow the success flow; invalid credentials are rejected.	Not recorded
T8	Refresh the booking list after a database change.	Display reflects backend/database data, not only sample data.	Not recorded

Table 3.2: Proposed functional and integration test plan. Update results only after running tests.

3.3 Evaluation and Current Limitations

The Oracle database is reported as connected, establishing a key part of the persistence layer.

The frontend, Java backend, and authentication remain partially implemented, so the current status does not yet establish that the complete application works end to end.

The main evaluation priority is to verify the full booking path and retain evidence for each test.

No response-time, throughput, or load-test measurements are reported because no such measurements have been provided. Add measured values only after running and recording the tests.

3.4 Challenges and Mitigation Plan

- **Endpoint mismatch:** Confirm frontend base URL, Tomcat context path, HTTP method, request body, and response format agree with Servlet mappings.
- **Deployment:** Confirm Maven creates the expected WAR and Tomcat deploys it successfully. Use server logs to investigate a browser 404.
- **Database consistency:** Verify the application uses the intended Oracle schema and that successful operations persist records.
- **Authentication:** Confirm session behavior is consistent across frontend navigation and protected backend endpoints.
- **Slot availability:** Reject unavailable slots and consider transaction/database safeguards against concurrent booking attempts.

These are verification priorities and potential risks, not claims that each issue has occurred or been resolved.

Chapter 4

Conclusion

4.1 Summary of Work

The Smart Campus Resource Booking System is under development as a web application combining a React frontend, Java backend, and Oracle database.

The team reports that the database is connected and that the frontend, backend, and authentication are partially implemented.

The project has established part of the technical foundation, while end-to-end integration and systematic validation remain important next steps.

4.2 Future Work

The planned work before final evaluation is:

1. Complete and verify Java backend build and deployment.
2. Confirm the Oracle schema and verify backend read/write operations.
3. Connect the frontend to real backend endpoints and remove or clearly separate prototype data.
4. Complete authentication and protected-operation checks.
5. Verify the full booking workflow, including invalid input and unavailable slots.
6. Record test evidence, add genuine application screenshots, and update the report to reflect the submitted implementation.

4.3 Final Remarks

The project is being developed collaboratively, with frontend and backend responsibilities separated but connected through a request-and-response contract.

The next milestone is a demonstrable, tested booking journey across the interface, Java application, and Oracle database.

The final report should be updated with verified screenshots, test outcomes, and implementation details from the submitted version.